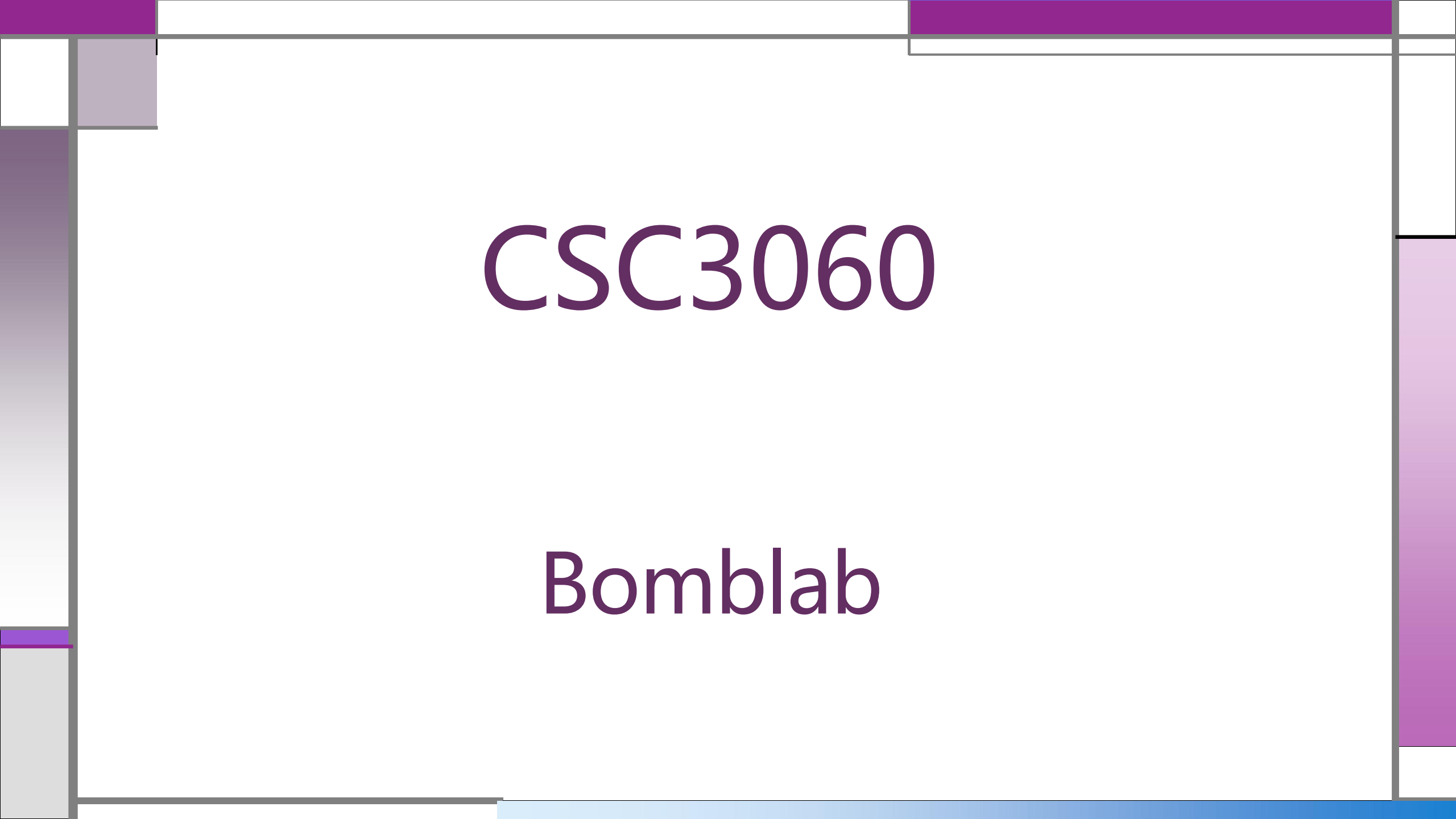




CSC3060

Bomblab

What is a bomb?

- An executable file consisting of a few phases.
- Each phase expects you to enter an input string.
- If your entered string is **correct**, the phase is defused.
- If it is wrong, the bomb goes off!

String correct,
first phase
defused

Prompts you
to enter string
for second
phase

String wrong,
the bomb
explodes

Program terminates,
and you fail

```
[root@csc3060 ~]# cd bomblab_prep/
[root@csc3060 ~/bomblab_prep]# ./bomb
```

Oh no! Prof.Hsu plans to spread his bombs everywhere!
And it will be up to YOU guys to defuse the bombs and save our machines!
I thereby provide you a practice bomb to ready you for Prof.Hsu's scheme.
It only has 2 phases to get you familiar with RISC-V assembly and gdb.

Oh no! Prof.Hsu plans to spread his bombs everywhere!
And it will be up to YOU guys to defuse the bombs and save our machines!
I thereby provide you a practice bomb to ready you for Prof.Hsu's scheme
It only has 2 phases to get you familiar with RISC-V assembly and gdb.

Some random string

[illegible]

The bomb has blown up.

```
[root@csc3060 ~]# cd bomblab_prep/
[root@csc3060 ~/bomblab_prep]# ./bomb
```

Oh no! Prof.Hsu plans to spread his bombs everywhere!
And it will be up to YOU guys to defuse the bombs and save our machines!
I thereby provide you a practice bomb to ready you for Prof.Hsu's scheme.
It only has 2 phases to get you familiar with RISC-V assembly and gdb.

Enter your
string here

But how the heck do you expect us to know what string is correct?

Remember. You have the bomb's executable file.
It has all you wish to know.

The GNU Debugger (GDB)

- A powerful debugging tool widely used for debugging programs written in C/C++.
- Available most commonly on Linux, but also on MAC, Windows, etc, as well.
- Architecture specific – A RISC-V executable file cannot run under x86 GDB.

```
[root@csc3060 ~/bomblab_prep]# gdb ./bomb
GNU gdb (GDB) 13.2
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "riscv32-buildroot-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./bomb...
(No debugging symbols found in ./bomb)
(gdb) |
```

Useful links:

[GDB Command Reference](#)

[GDB manual: Debugging with GDB](#)

[100 GDB tips \(In Chinese\)](#)

Frequently used GDB commands

- `break <function_name>` / `break <line>` / `break *<address>`: (abbr: `b`)

to set breakpoints: points in the program's execution where you can temporarily pause the program and inspect its state. You can also quit the program when it is paused at a breakpoint.

- `run`: (abbr: `r`)

to run the program from the beginning and stop at the first breakpoint.

- `continue`: (abbr: `c`)

to continue running the program until the next breakpoint.

- `step` / `stepi`: (abbr: `s`)

to step into the next line / instruction of the code. Used to follow the program's logic line by line if the current piece of code looks as if it is doing nonsense to you.

Frequently used GDB commands

- `finish`: (abbr: `f`)

to finish executing the current function if you are inside one.

- `info registers`: (abbr: `i r`)

to display the current contents of all the registers used in the program (x1-x31 + pc). There are other "info" commands as well, such as "info locals" and "info frame", but this may be too advanced for this lab (feel free to check them out if interested).

- `print $<register>`: (abbr: `p`)

to print a specific register's content (usually in decimal). Use `print/x` for hexadecimal.

- `x / [length] [format] [address]`:

to "x" (examine) the contents of a specific address. Formats include: `x` – hex, `s` – str, `c` – char, `a` – addr, `w` – word, `h` – halfword, etc. The address can be a specific one such as `x/32s 0x5555555555555555` or one residing in a register such as `x/32s $a0`.

Phase 1

```
Breakpoint 1, 0x00010690 in phase_1 ()
(gdb) disas
Dump of assembler code for function phase_1:
0x00010686 <+0>:      add     sp,sp,-16
0x00010688 <+2>:      sw      ra,12(sp)
0x0001068a <+4>:      lui     a1,0x11
0x0001068c <+6>:      add     a1,a1,-1184 # 0x10b60
=> 0x00010690 <+10>:   jal     0x10540 <strcmp@plt>
0x00010692 <+12>:   bnez    a0,0x1069a <phase_1+20>
0x00010694 <+14>:   lw      ra,12(sp)
0x00010696 <+16>:   add     sp,sp,16
0x00010698 <+18>:   ret
0x0001069a <+20>:   jal     0x1079e <explode_bomb>
0x0001069c <+22>:   j       0x10694 <phase_1+14>
```

Returned
in a0

result = strcmp (str1, str2);

Address
stored
in a0

Address
stored
in a1

Look what we
have here...

Ooops!
We don't want
this to happen!

- How does RISC-V pass its function arguments?

We use the registers a0, a1, a2, ... a7. If it's still not enough, we use the stack.

- How does a function return its values?

Still a0, a1, ... a7.

Phase 2

```
Breakpoint 1, 0x000106ae in phase_2 ()
(gdb) disas
Dump of assembler code for function phase_2:
0x0001069e <+0>:      add     sp,sp,-48
0x000106a0 <+2>:      sw      ra,44(sp)
0x000106a2 <+4>:      sw      s0,40(sp)
0x000106a4 <+6>:      sw      s1,36(sp)
0x000106a6 <+8>:      sw      s2,32(sp)
0x000106a8 <+10>:     sw      s3,28(sp)
0x000106aa <+12>:     sw      s4,24(sp)
0x000106ac <+14>:     mv      a1,sp
=> 0x000106ae <+16>:     jal     0x108c6 <read_four_ints>
```

```
Breakpoint 2, 0x000108de in read_four_ints ()
(gdb) disas
Dump of assembler code for function read_four_ints:
0x000108c6 <+0>:      add     sp,sp,-16
0x000108c8 <+2>:      sw      ra,12(sp)
0x000108ca <+4>:      mv      a2,a1
0x000108cc <+6>:      add     a5,a1,12
0x000108d0 <+10>:     add     a4,a1,8
0x000108d4 <+14>:     add     a3,a1,4
0x000108d8 <+18>:     lui      a1,0x11
0x000108da <+20>:     add     a1,a1,-536 # 0x10de8
=> 0x000108de <+24>:     jal     0x104c0 <__isoc23_sscanf@plt>
0x000108e0 <+26>:     li      a5,4
0x000108e2 <+28>:     bne     a0,a5,0x108ec <read_four_ints+38>
0x000108e6 <+32>:     lw      ra,12(sp)
0x000108e8 <+34>:     add     sp,sp,16
0x000108ea <+36>:     ret
0x000108ec <+38>:     jal     0x1079e <explode_bomb>
End of assembler dump.
```

Returned
in a0

num_inputs = sscanf(input, format, &addr1, &addr2, ...);

Address
stored
in a0

Address
stored
in a1

Address
stored
in a2

Address
stored
in a3

It called a
function. Let's
start here.

What is hidden
in this address?

```
(gdb) x/s $a1
0x10de8:      "%d %d %d %d"
```

Oooh! So we need to input four ints!
...which is kinda obvious...
But where are they stored?

Phase 2

```
=> 0x000106ac <+14>: mv      a1,sp
    0x000106ae <+16>: jal      0x108c6 <read_four_ints>
    0x000106b0 <+18>: lw       s1,0(sp)
    0x000106b2 <+20>: lw       s2,4(sp)
    0x000106b4 <+22>: lw       s0,8(sp)
    0x000106b6 <+24>: lw       s3,12(sp)
    0x000106b8 <+26>: add      a5,s1,-10
    0x000106bc <+30>: bnez     a5,0x106d8 <phase_2+58>
    0x000106be <+32>: add      a5,s2,-12
    0x000106c2 <+36>: bnez     a5,0x106d8 <phase_2+58>
    0x000106c4 <+38>: li       a5,5
    0x000106c6 <+40>: bltu     a5,s0,0x10712 <phase_2+116>
    0x000106ca <+44>: sll      s0,s0,0x2
    0x000106cc <+46>: lui      a5,0x11
    0x000106ce <+48>: add      a5,a5,-472 # 0x10e28
    0x000106d2 <+52>: add      s0,s0,a5
    0x000106d4 <+54>: lw       a5,0(s0)
    0x000106d6 <+56>: jr       a5
    0x000106d8 <+58>: jal      0x1079e <explode_bomb>
```

s1: first input
s2: second input
s0: third input
s3: forth input

s1 must be 10

s2 must be 12

s0 must be ≥ 0 and < 5

Phase 2

If we input:
10 12 0 ?

```
0x000106c2 <+36>:    bnez    a5,0x106d8 <phase_2+58>
0x000106c4 <+38>:    li      a5,5
0x000106c6 <+40>:    bltu    a5,s0,0x10712 <phase_2+116>
0x000106ca <+44>:    sll     s0,s0,0x2
0x000106cc <+46>:    lui     a5,0x11
0x000106ce <+48>:    add     a5,a5,-472 # 0x10e28
0x000106d2 <+52>:    add     s0,s0,a5
0x000106d4 <+54>:    lw      a5,0(s0)
=> 0x000106d6 <+56>:    jr      a5
0x000106d8 <+58>:    jal     0x1079e <explode_bomb>
0x000106da <+60>:    j       0x106c4 <phase_2+38>
0x000106dc <+62>:    add     s4,s1,s2
--Type <RET> for more, q to quit, c to continue without paging--RET
0x000106e0 <+66>:    bne     s4,s3,0x10716 <phase_2+120>
```

s3 = s1+s2=22

```
(gdb) x/a $a5
```

```
0x106dc <phase_2+62>:    0x1248a33
```

Now we can conclude that the input is
10 12 0 22

Phase 2

If we input:
10 12 1 ?

```
=> 0x000106d6 <+56>: jr      a5
0x000106d8 <+58>: jal     0x1079e <explode_bomb>
0x000106da <+60>: j       0x106c4 <phase_2+38>
0x000106dc <+62>: add     s4,s1,s2
--Type <RET> for more, q to quit, c to continue without paging--
0x000106e0 <+66>: bne     s4,s3,0x10716 <phase_2+120>
0x000106e4 <+70>: lw      ra,44(sp)
0x000106e6 <+72>: lw      s0,40(sp)
0x000106e8 <+74>: lw      s1,36(sp)
0x000106ea <+76>: lw      s2,32(sp)
0x000106ec <+78>: lw      s3,28(sp)
0x000106ee <+80>: lw      s4,24(sp)
0x000106f0 <+82>: add     sp,sp,48
0x000106f2 <+84>: ret
0x000106f4 <+86>: sub     s4,s1,s2
0x000106f8 <+90>: j       0x106e0 <phase_2+66>
0x000106fa <+92>: mul     s4,s1,s2
0x000106fe <+96>: j       0x106e0 <phase_2+66>
0x00010700 <+98>: and     s4,s1,s2
0x00010704 <+102>: j       0x106e0 <phase_2+66>
0x00010706 <+104>: or      s4,s1,s2
0x0001070a <+108>: j       0x106e0 <phase_2+66>
0x0001070c <+110>: xor     s4,s1,s2
0x00010710 <+114>: j       0x106e0 <phase_2+66>
0x00010712 <+116>: jal     0x1079e <explode_bomb>
0x00010714 <+118>: j       0x106e0 <phase_2+66>
0x00010716 <+120>: jal     0x1079e <explode_bomb>
0x00010718 <+122>: j       0x106e4 <phase_2+70>
End of assembler dump.
(gdb) x/a $a5
0x106f4 <phase_2+86>: 0x41248a33
(gdb)
```

$s3 = s1 - s2 = -2$

Now we can conclude that the input can also be
10 12 1 -2



Thank you

Hopefully now you are better prepared
for the bomb squad.

